

1 Introduction to R and RStudio

Mohan Khanal

Table of contents

1	Welcome to R Programming!	2
1.1	What is R?	2
1.2	Why Learn R?	2
2	Getting Started with RStudio	3
2.1	RStudio Interface Overview	3
2.1.1	Quick Tips:	3
3	Your First R Code	3
3.1	The Classic “Hello World”	3
3.2	Built-in Constants	3
4	Basic Math Operations	4
4.1	Arithmetic Operators	4
4.2	Order of Operations (PEMDAS)	5
5	Data Types in R	5
5.1	1. Numeric (Double)	5
5.2	2. Integer	6
5.3	3. Character (String)	7
5.4	4. Logical (Boolean)	8
5.5	Quick Type Check Summary	9
6	Variables and Assignment	9
6.1	Creating Variables	9
6.2	Variable Naming Rules	10
6.3	Best Practices	11
7	Using Functions	11
7.1	What are Functions?	11

7.2	Basic Mathematical Functions	11
7.3	The c() Function (Concatenate)	12
7.4	Statistical Functions	13
8	Sequences and Patterns	14
8.1	Creating Sequences	14
9	Getting Help in R	15
9.1	Help Functions	15
9.2	Useful Commands	16
10	Practice Exercises	17
10.1	Exercise 1: Basic Calculations	17
10.2	Exercise 2: Working with Vectors	17
10.3	Exercise 3: Data Types	18
11	Key Takeaways	19
12	Homework / Practice	19
13	Next Session Preview	20
14	Resources	20

1 Welcome to R Programming!

1.1 What is R?

R is a powerful programming language and environment for:

- Statistical computing and analysis
- Data visualization
- Data manipulation and cleaning
- Machine learning and predictive modeling
- Reproducible research

1.2 Why Learn R?

- **Free and Open Source:** No licensing costs
- **Extensive Packages:** Over 19,000 packages available
- **Strong Community:** Active support and resources
- **Industry Standard:** Used in academia, healthcare, finance, tech

- **Data Science Ready:** Built specifically for data analysis
-

2 Getting Started with RStudio

2.1 RStudio Interface Overview

RStudio has 4 main panes:

1. **Source/Editor Pane** (Top Left): Where you write and save your code
2. **Console Pane** (Bottom Left): Where code executes and shows output
3. **Environment/History Pane** (Top Right): Shows variables and command history
4. **Files/Plots/Packages/Help Pane** (Bottom Right): Multiple tabs for different functions

2.1.1 Quick Tips:

- Press **Ctrl + Enter** (Windows) or **Cmd + Enter** (Mac) to run current line
 - Press **Ctrl + Shift + Enter** to run entire script
 - Use **#** for comments (code that won't execute)
-

3 Your First R Code

3.1 The Classic “Hello World”

```
print("Hello World")
```

```
[1] "Hello World"
```

3.2 Built-in Constants

```
# Pi is a built-in constant in R
pi
```

```
[1] 3.141593
```

```
# R has many mathematical constants
exp(1) # Euler's number (e)
```

```
[1] 2.718282
```

4 Basic Math Operations

4.1 Arithmetic Operators

```
# Addition
2 + 2
```

```
[1] 4
```

```
# Subtraction
10 - 3
```

```
[1] 7
```

```
# Multiplication
4 * 5
```

```
[1] 20
```

```
# Division
20 / 4
```

```
[1] 5
```

```
# Exponentiation (Power)
2^3
```

```
[1] 8
```

```
# Modulo (Remainder)
10 %% 3
```

```
[1] 1
```

```
# Integer Division
10 %/% 3
```

```
[1] 3
```

4.2 Order of Operations (PEMDAS)

```
# R follows standard mathematical order
2 + 3 * 4      # Multiplication first
```

```
[1] 14
```

```
# Use parentheses to change order
(2 + 3) * 4     # Addition first
```

```
[1] 20
```

5 Data Types in R

5.1 1. Numeric (Double)

The default for numbers with decimals:

```
x <- 5  
x
```

```
[1] 5
```

```
class(x)
```

```
[1] "numeric"
```

```
typeof(x)
```

```
[1] "double"
```

```
# Decimal numbers  
m <- 0.03  
class(m)
```

```
[1] "numeric"
```

```
# Scientific notation  
large_num <- 1.5e6 # 1,500,000  
large_num
```

```
[1] 1500000
```

5.2 2. Integer

Whole numbers (add 'L' suffix):

```
y <- 444L  
class(y)
```

```
[1] "integer"
```

```
typeof(y)
```

```
[1] "integer"
```

```
# Check if it's an integer
is.integer(y)
```

```
[1] TRUE
```

```
is.integer(5)      # FALSE - default is numeric
```

```
[1] FALSE
```

```
is.integer(5L)     # TRUE
```

```
[1] TRUE
```

5.3 3. Character (String)

Text data enclosed in quotes:

```
z <- "This is our first data camp"
z
```

```
[1] "This is our first data camp"
```

```
class(z)
```

```
[1] "character"
```

```
# Single or double quotes both work
name <- 'John'
greeting <- "Hello, World!"

# Combine strings
paste("Hello", "R", "Programming")
```

```
[1] "Hello R Programming"
```

```
paste0("Hello", "World") # No space
```

```
[1] "HelloWorld"
```

5.4 4. Logical (Boolean)

TRUE or FALSE values:

```
aa <- FALSE  
class(aa)
```

```
[1] "logical"
```

```
bb <- TRUE  
bb
```

```
[1] TRUE
```

```
# Logical operations  
TRUE & FALSE # AND
```

```
[1] FALSE
```

```
TRUE | FALSE # OR
```

```
[1] TRUE
```

```
!TRUE # NOT
```

```
[1] FALSE
```

```
# Comparisons return logical values  
5 > 3
```

```
[1] TRUE
```



```
10 == 10
```

```
[1] TRUE
```

```
7 != 8
```

```
[1] TRUE
```

5.5 Quick Type Check Summary

```
# Check multiple types at once  
check_value <- 42  
  
is.numeric(check_value)
```

```
[1] TRUE
```

```
is.character(check_value)
```

```
[1] FALSE
```

```
is.logical(check_value)
```

```
[1] FALSE
```

6 Variables and Assignment

6.1 Creating Variables

```
# Use <- or = for assignment (+ is preferred in R)  
k <- 11  
l <- 9  
  
# View variables  
k
```

```
[1] 11
```

```
l
```

```
[1] 9
```

```
# Use variables in calculations  
k + l
```

```
[1] 20
```

```
k * l
```

```
[1] 99
```

```
k / l
```

```
[1] 1.222222
```

```
# Update variables  
k <- k + 5  
k
```

```
[1] 16
```

6.2 Variable Naming Rules

```
# Good variable names  
student_age <- 20  
studentAge <- 20      # camelCase  
student.age <- 20     # dot notation  
  
# Names can include numbers (but not start with them)  
data1 <- 100  
# 1data <- 100 # This would cause an ERROR  
  
# Case sensitive  
Value <- 10  
value <- 20  
Value == value # FALSE
```

```
[1] FALSE
```

6.3 Best Practices

- Use descriptive names: `student_score` instead of `x`
 - Be consistent with naming style
 - Avoid special characters except `.` and `_`
 - Don't overwrite built-in functions (like `mean`, `sum`, `c`)
-

7 Using Functions

7.1 What are Functions?

Functions are pre-built commands that perform specific tasks.

Syntax: `function_name(arguments)`

7.2 Basic Mathematical Functions

```
# Square root  
sqrt(16)
```

```
[1] 4
```

```
# Absolute value  
abs(-5)
```

```
[1] 5
```

```
# Rounding  
round(3.14159, 2) # Round to 2 decimal places
```

```
[1] 3.14
```

```
# Logarithms  
log(10) # Natural log
```

```
[1] 2.302585
```

```
log10(100)    # Base 10 log
```

```
[1] 2
```

```
# Exponential  
exp(2)
```

```
[1] 7.389056
```

7.3 The c() Function (Concatenate)

Creates vectors (collections of values):

```
# Create a vector of numbers  
numbers <- c(4, 5, 6)  
numbers
```

```
[1] 4 5 6
```

```
# Calculate mean  
mean(c(4, 5, 6))
```

```
[1] 5
```

```
# Calculate sum  
sum(c(4, 5, 6, 7))
```

```
[1] 22
```

```
# Create sequences  
sum(1:10)      # Sum of 1 to 10
```

```
[1] 55
```

```
mean(1:10)      # Mean of 1 to 10
```

```
[1] 5.5
```

```
# More with vectors  
ages <- c(23, 45, 67, 34, 29)  
min(ages)
```

```
[1] 23
```

```
max(ages)
```

```
[1] 67
```

```
median(ages)
```

```
[1] 34
```

```
length(ages)    # How many elements
```

```
[1] 5
```

7.4 Statistical Functions

```
# Create sample data  
scores <- c(85, 90, 78, 92, 88, 76, 95)  
  
# Descriptive statistics  
mean(scores)      # Average
```

```
[1] 86.28571
```

```
median(scores)    # Middle value
```

```
[1] 88
```

```
sd(scores)          # Standard deviation
```

```
[1] 7.087884
```

```
var(scores)         # Variance
```

```
[1] 50.2381
```

```
range(scores)       # Min and max
```

```
[1] 76 95
```

```
summary(scores)     # Quick summary
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
76.00	81.50	88.00	86.29	91.00	95.00

8 Sequences and Patterns

8.1 Creating Sequences

```
# Using colon operator
```

```
1:10
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
5:15
```

```
[1] 5 6 7 8 9 10 11 12 13 14 15
```

```
10:1 # Descending
```

```
[1] 10 9 8 7 6 5 4 3 2 1
```

```
# Using seq() function
seq(1, 10) # Same as 1:10
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
seq(0, 100, by = 10) # Specify step
```

```
[1] 0 10 20 30 40 50 60 70 80 90 100
```

```
seq(0, 1, by = 0.1) # Decimal steps
```

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
seq(0, 1, length.out = 11) # Specify length
```

```
[1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
# Repeating values
rep(5, times = 3) # Repeat 5 three times
```

```
[1] 5 5 5
```

```
rep(1:3, times = 3) # Repeat sequence
```

```
[1] 1 2 3 1 2 3 1 2 3
```

```
rep(1:3, each = 3) # Repeat each element
```

```
[1] 1 1 1 2 2 2 3 3 3
```

9 Getting Help in R

9.1 Help Functions

```

# Get help on a function
?mean
help(sum)

# Search for topics
??regression
help.search("linear model")

# Examples of function usage
example(mean)

# See arguments of a function
args(seq)

```

9.2 Useful Commands

```

# List objects in environment
ls()

```

```

[1] "aa"           "ages"         "bb"           "check_value"
[5] "data1"        "greeting"     "k"            "l"
[9] "large_num"    "m"           "name"         "numbers"
[13] "pandoc_dir"   "quarto_bin_path" "scores"       "student_age"
[17] "student.age"  "studentAge"    "value"        "Value"
[21] "x"           "y"           "z"

```

```

# Remove an object
# rm(x)

# Clear entire environment
# rm(list = ls())

# Check current working directory
getwd()

```

```

[1] "/home/mk/Documents/khanalmohan.github.io/r-tutorial"

```



```
# Set working directory (example - modify path)
# setwd("C:/Users/YourName/Documents/R_Training")
```

10 Practice Exercises

10.1 Exercise 1: Basic Calculations

Calculate the following and store in variables:

1. The sum of 15 and 27
2. 8 raised to the power of 3
3. The square root of 144

```
# Your code here
sum_result <- 15 + 27
power_result <- 8^3
sqrt_result <- sqrt(144)
```

```
# View results
sum_result
```

```
[1] 42
```

```
power_result
```

```
[1] 512
```

```
sqrt_result
```

```
[1] 12
```

10.2 Exercise 2: Working with Vectors

Create a vector of temperatures: 72, 68, 75, 80, 77

Calculate: - Mean temperature - Highest temperature - Temperature range

```
# Your code here
temperatures <- c(72, 68, 75, 80, 77)

mean(temperatures)
```

```
[1] 74.4
```

```
max(temperatures)
```

```
[1] 80
```

```
range(temperatures)
```

```
[1] 68 80
```

10.3 Exercise 3: Data Types

Identify the data type of the following:

```
var1 <- 42
var2 <- "Data Science"
var3 <- TRUE
var4 <- 3.14159

class(var1)
```

```
[1] "numeric"
```

```
class(var2)
```

```
[1] "character"
```

```
class(var3)
```

```
[1] "logical"
```

```
class(var4)
```

```
[1] "numeric"
```

11 Key Takeaways

1. **R is a powerful tool** for data analysis and statistics
 2. **RStudio** provides an excellent interface for R programming
 3. **Data types** matter: numeric, integer, character, logical
 4. **Variables** store values using `<-` operator
 5. **Functions** perform operations: `mean()`, `sum()`, `c()`, etc.
 6. **Vectors** are collections of values created with `c()`
 7. **Help is available** using `?function_name`
-

12 Homework / Practice

Before our next session, practice:

1. Install R and RStudio on your computer (if not already done)
 2. Create 5 different variables with different data types
 3. Perform at least 10 different mathematical operations
 4. Create 3 vectors and calculate their mean, median, and standard deviation
 5. Experiment with the `seq()` and `rep()` functions
 6. Use the `?` operator to explore 3 new functions
-

13 Next Session Preview

January 07, 2026: Data Import and Data Wrangling

We'll cover: - Reading CSV, Excel, and other data files - Introduction to the `dplyr` package - Data cleaning and manipulation techniques - Handling missing data (NA values)

Prepare: Make sure you have a CSV file ready to import (any data will work!)

14 Resources

- [R Documentation](#)
 - [RStudio Cheatsheets](#)
 - [R for Data Science \(Free Book\)](#)
 - [Stack Overflow - R Tag](#)
-

Questions? Feel free to ask during the session or practice at home!

Happy Coding!